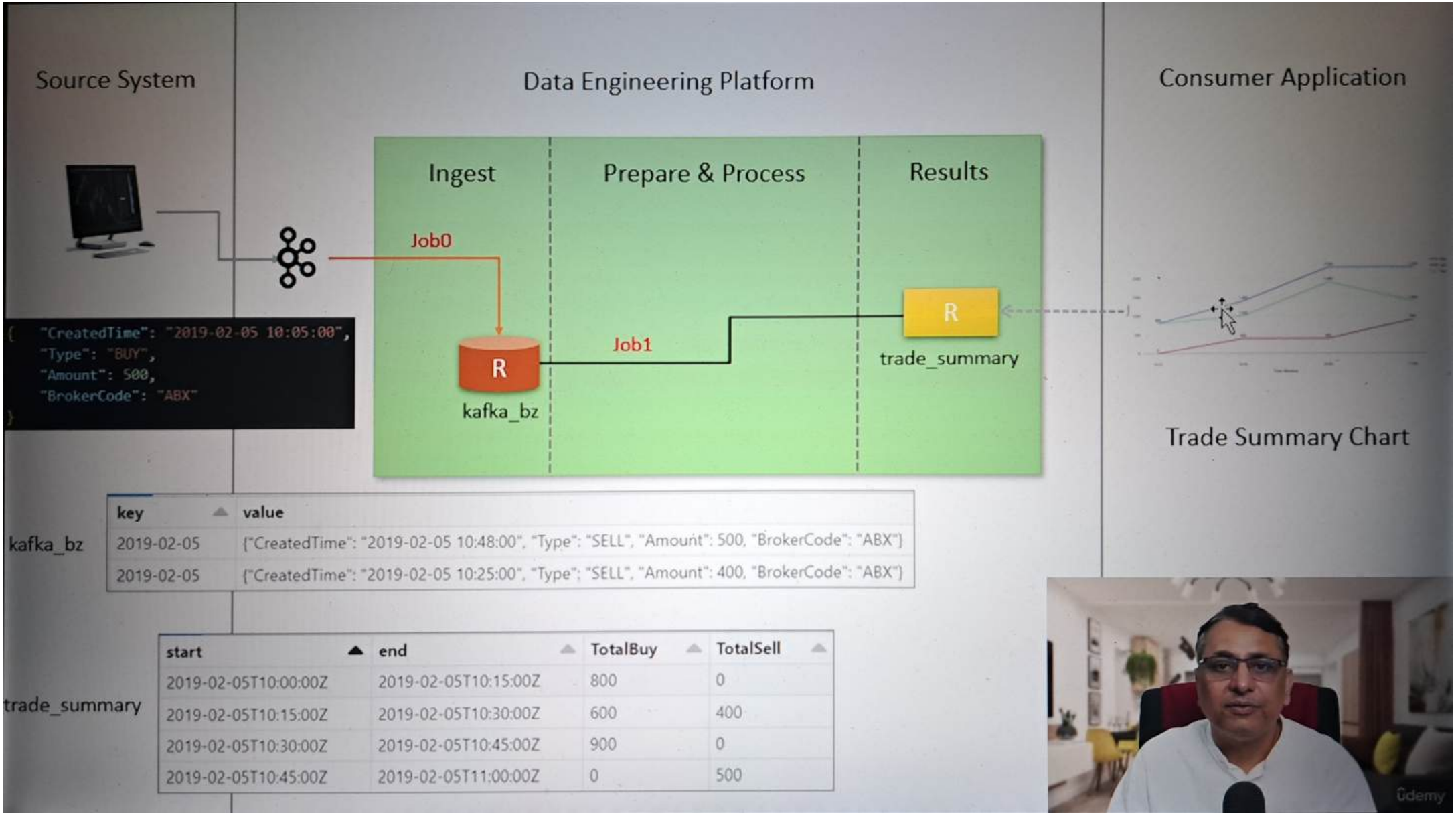
What is a time-bound aggregate?

- Popularly known as window aggregates
- Window is a time window
- Window size could be microseconds, milliseconds, seconds, minutes, hours, and days
- You need a timestamp column (event time) in your input records



kafka_bz

key	value
2019-02-05	{"CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 500, "BrokerCode": "ABX"}
2019-02-05	{"CreatedTime": "2019-02-05 10:25:00", "Type": "SELL", "Amount": 400, "BrokerCode": "ABX"}

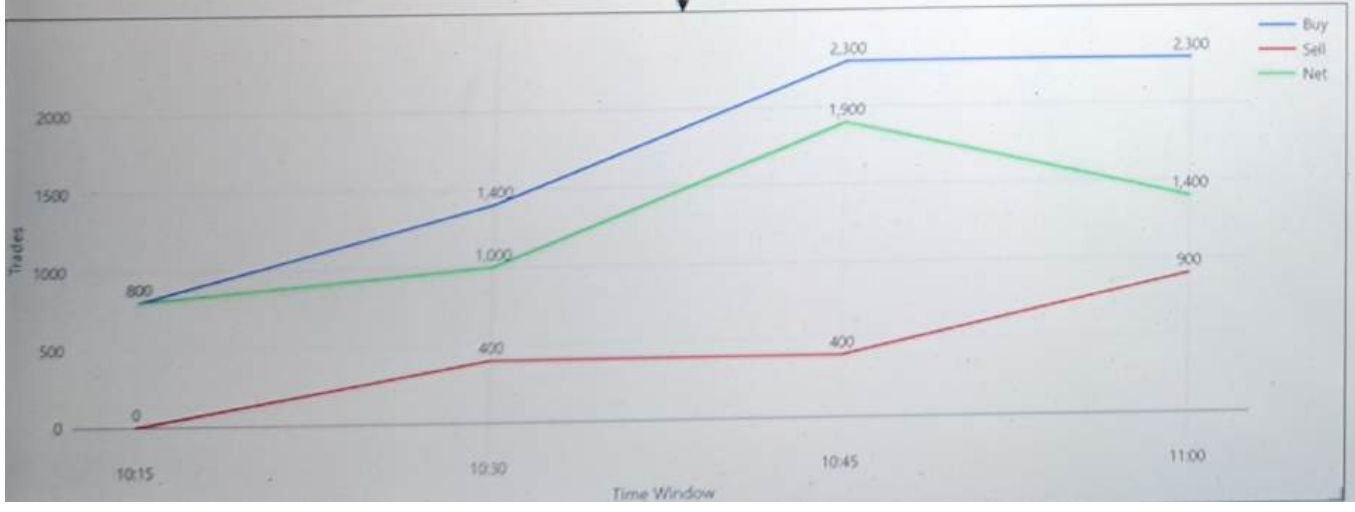
trade_summary

start	end	TotalBuy	TotalSell
2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	400
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0
2019-02-05T10:45:00Z	2019-02-05T11:00:00Z	0	500

at_time	Buy	Sell	Net
10:15	800	0	800
10:30	1400	400	1000
10:45	2300	400	1900
11:00	2300	900	1400

start	end	TotalBuy	TotalSell
2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	400
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0
2019-02-05T10:45:00Z	2019-02-05T11:00:00Z	0	500

```
select date_format(end, 'HH:mm') as at_time,
       sum(TotalBuy) over (order by end) as Buy,
       sum(TotalSell) over (order by end) as Sell,
       Buy - Sell as Net
from trade_summary
```



22-tumbling-time-window Python

File Edit View Run Help Last edit was 3 minutes ago Provide feedback

Run all Connect Publish

Cmd 1

Python

```
class TradeSummary():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def getSchema(self):
        from pyspark.sql.types import StructType, StructField, StringType, DoubleType
        return StructType([
                StructField("CreatedTime", StringType()),
                StructField("Type", StringType()),
                StructField("Amount", DoubleType()),
                StructField("BrokerCode", StringType())
            ])

    def readBronze(self):
        return spark.readStream.table("kafka_bz")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

22-tumbling-time-window Python

File Edit View Run Help Last edit was 14 minutes ago Provide feedback

Run all Connect Share Publish

```
    def getTrade(self, kafka_df):
        from pyspark.sql.functions import from_json, expr
        return (kafka_df.select(from_json(kafka_df.value, self.getSchema()).alias("value"))
                        .select("value.*")
                        .withColumn("CreatedTime", expr("to_timestamp(CreatedTime, 'yyyy-MM-dd HH:mm:ss')"))
                        .withColumn("Buy", expr("case when Type == 'BUY' then Amount else 0 end"))
                        .withColumn("Sell", expr("case when Type == 'SELL' then Amount else 0 end"))
                )
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

```
    def getAggregate(self, trade_df):
        from pyspark.sql.functions import window, sum
        return (trade_df.groupBy(window(trade_df.CreatedTime, "15 minutes"))
                        .agg(sum("Buy").alias("TotalBuy"),
                             sum("Sell").alias("TotalSell"))
                        .select("window.start", "window.end", "TotalBuy", "TotalSell")
                )
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

```
    def saveResults(self, results_df):
        print(f"\nStarting Silver Stream...", end='')
        return (results_df.writeStream
                          .queryName("trade-summary")
                          .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/trade_summary")
                          .outputMode("complete")
                          .toTable("trade_summary")
                )
        print("Done")

    def process(self):
        kafka_df = self.readBronze()
        trade_df = self.getTrade(kafka_df)
        result_df = self.getAggregate(trade_df)
        self.saveResults(result_df)
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

22-tumbling-time-window Python

File Edit View Run Help Last edit was 26 minutes ago Provide feedback

Run all Connect Share Publish

```
                .outputMode("complete")
                .toTable("trade_summary")
            )
        print("Done")

    def process(self):
        kafka_df = self.readBronze()
        trade_df = self.getTrade(kafka_df)
        result_df = self.getAggregate(trade_df)
        sQuery = self.saveResults(result_df)
        return sQuery
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

```
{"CreatedTime": "2019-02-05 10:05:00", "Type": "BUY", "Amount": 500, "BrokerCode": "ABX"}
{"CreatedTime": "2019-02-05 10:12:00", "Type": "BUY", "Amount": 300, "BrokerCode": "ABX"}
{"CreatedTime": "2019-02-05 10:20:00", "Type": "BUY", "Amount": 600, "BrokerCode": "ABX"}
{"CreatedTime": "2019-02-05 10:40:00", "Type": "BUY", "Amount": 900, "BrokerCode": "ABX"}
{"CreatedTime": "2019-02-05 10:25:00", "Type": "SELL", "Amount": 400, "BrokerCode": "ABX"}
{"CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 500, "BrokerCode": "ABX"}
```


start	end	TotalBuy	TotalSell
2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	
2019-02-05T10:45:00Z	2019-02-05T11:00:00Z	0	

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Publish

```
class TradeSummaryTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists kafka_bz")
        spark.sql("drop table if exists trade_summary")
        dbutils.fs.rm("/user/hive/warehouse/kafka_bz", True)
        dbutils.fs.rm("/user/hive/warehouse/trade_summary", True)
        spark.sql(f"CREATE TABLE kafka_bz(key STRING, value STRING)")

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/trade_summary", True)
        print("Done")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

```
        print("Done")

    def waitForMicroBatch(self, sleep=30):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was 22 minutes ago Provide feedback

Run all Connect Share Publish

```
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")

    def assertTradeSummary(self, start, end, expected_buy, expected_sell):
        print(f"\tStarting Trade Summary validation...", end='')
        result = (spark.sql(f"""select TotalBuy, TotalSell from trade_summary
                                   where date_format(start, 'yyyy-MM-dd HH:mm:ss') = '{start}'
                                   and date_format(end, 'yyyy-MM-dd HH:mm:ss')='{end}'""")
                        .collect()
                  )
        actual_buy = result[0][0]
        actual_sell = result[0][1]
        assert expected_buy == actual_buy, f"Test failed! actual buy is {actual_buy}"
        assert expected_sell == actual_sell, f"Test failed! actual sell is {actual_sell}"
        print("Done")
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was 23 minutes ago Provide feedback

Run all Connect Share Publish

```
        assert expected_buy == actual_buy, f"Test failed! actual buy is {actual_buy}"
        assert expected_sell == actual_sell, f"Test failed! actual sell is {actual_sell}"
        print("Done")

    def runTests(self):
        self.cleanTests()

        stream = TradeSummary()
        sQuery = stream.process()

        print("\nTesting first two events...")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was 25 minutes ago Provide feedback

Run all Connect Share Publish

```
        print("\nTesting first two events...")
        spark.sql("""INSERT INTO kafka_bz VALUES
                ('2019-02-05', '{"CreatedTime": "2019-02-05 10:05:00", "Type": "BUY", "Amount": 500, "BrokerCode": "ABX"}'),
                ('2019-02-05', '{"CreatedTime": "2019-02-05 10:12:00", "Type": "BUY", "Amount": 300, "BrokerCode": "ABX"}')
            """)
        self.waitForMicroBatch()
        self.assertTradeSummary('2019-02-05 10:00:00', '2019-02-05 10:15:00', 800, 0)
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

```
        self.waitForMicroBatch()
        self.assertTradeSummary('2019-02-05 10:00:00', '2019-02-05 10:15:00', 800, 0)

        print("\nTesting third and fourth events...")
        spark.sql("""INSERT INTO kafka_bz VALUES
                    ('2019-02-05', '{"CreatedTime": "2019-02-05 10:20:00", "Type": "BUY", "Amount": 600, "BrokerCode": "ABX"}'),
                    ('2019-02-05', '{"CreatedTime": "2019-02-05 10:40:00", "Type": "BUY", "Amount": 900, "BrokerCode": "ABX"}')
            """)
        self.waitForMicroBatch()
        self.assertTradeSummary('2019-02-05 10:15:00', '2019-02-05 10:30:00', 600, 0)
        self.assertTradeSummary('2019-02-05 10:30:00', '2019-02-05 10:45:00', 900, 0)

        print("\nTesting late event...")
        spark.sql("""INSERT INTO kafka_bz VALUES
                    ('2019-02-05', '{"CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 500, "BrokerCode": "ABX"}'),
                    ('2019-02-05', '{"CreatedTime": "2019-02-05 10:25:00", "Type": "SELL", "Amount": 400, "BrokerCode": "ABX"}')
            """)
        self.waitForMicroBatch()
        self.assertTradeSummary('2019-02-05 10:45:00', '2019-02-05 11:00:00', 0, 500)
        self.assertTradeSummary('2019-02-05 10:15:00', '2019-02-05 10:30:00', 600, 400)

        print("Validation passed.\n")

        sQuery.stop()
```

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was now Provide feedback

Run all | my cluster | Share | Publish

SQL

```
%sql
select date_format(end, 'HH:mm') as at_time,
    sum(TotalBuy) over (order by end) as Buy,
    sum(TotalSell) over (order by end) as Sell,
    Buy - Sell as Net
from trade_summary
```

▸ (2) Spark Jobs

▸ _sqldf: pyspark.sql.dataframe.DataFrame = [at_time: string, Buy: double ... 2 more fields]

Table +

	at_time	Buy	Sell	Net
1	10:15	800	0	800
2	10:30	1400	400	1000
3	10:45	2300	400	1900
4	11:00	2300	900	1400

4 rows | 6.20 seconds runtime

This result is stored as PySpark data frame `_sqldf` and in the IPython output cache as `Out[8]`. Learn more

Command took 6.20 seconds -- by prashantkumarpandey@gmail.com at 11/6/2023, 5:53:55 PM on my cluster

23-tumbling-time-window-test-suite Python

File Edit View Run Help Last edit was 3 minutes ago Provide feedback

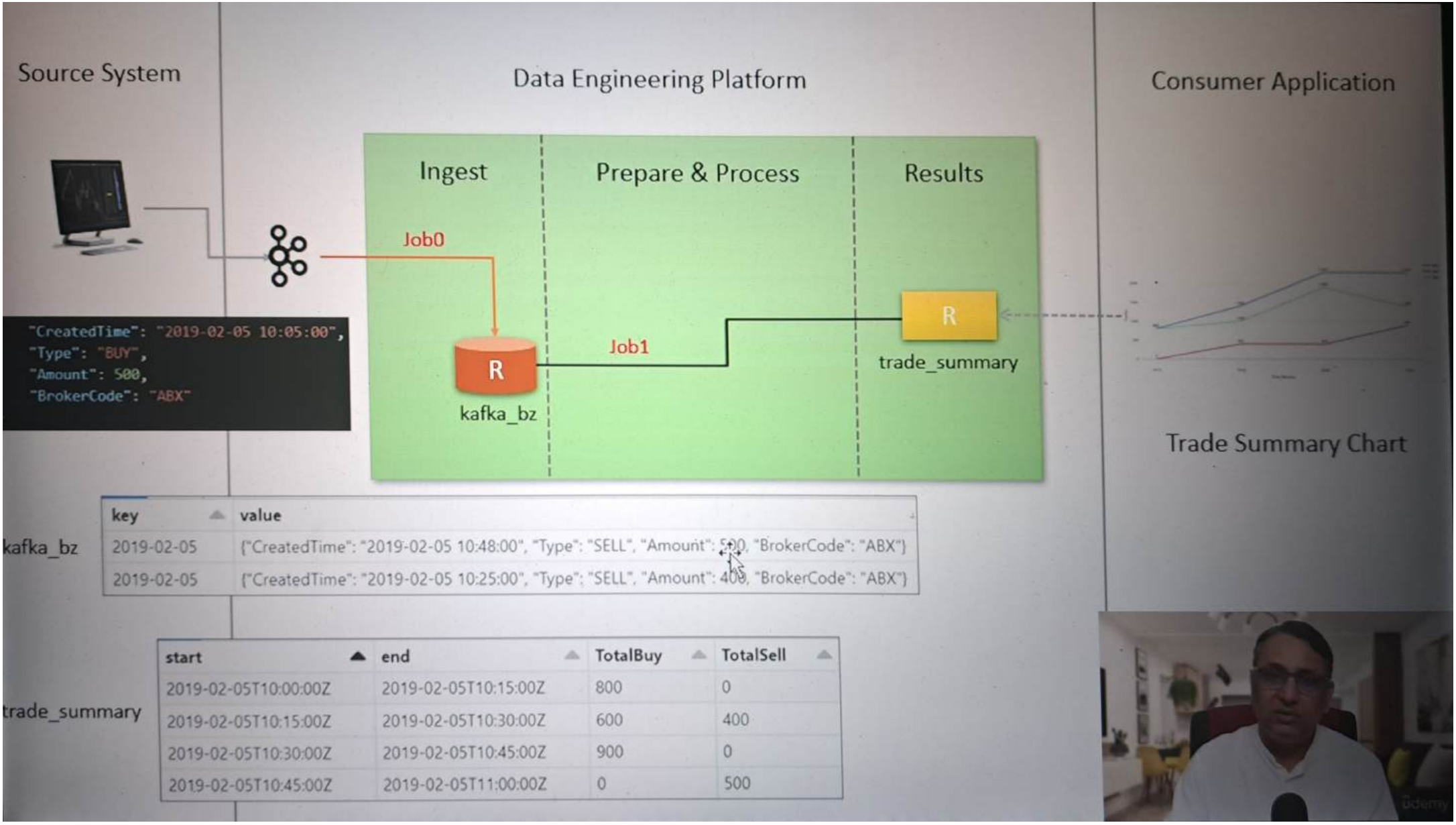
Run all my cluster Share Publish

Table Visualization 1 +

Edit Visualization 4 rows

This result is stored as PySpark data frame `_sqldf` and in the IPython output cache as `Out[8]` . Learn more

Command took 6.20 seconds -- by prashantkumarpandey@gmail.com at 11/6/2023, 5:53:55 PM on my cluster


Source System
Data Engineering Platform
Consumer Application
Ingest
Prepare & Process
Results
Job0
Job1
R
kafka_bz
R
trade_summary
"CreatedTime": "2019-02-05 10:05:00",
"Type": "BUY",
"Amount": 500,
"BrokerCode": "ABX"
Trade Summary Chart
kafka_bz
key
value
2019-02-05
{"CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 500, "BrokerCode": "ABX"}
2019-02-05
{"CreatedTime": "2019-02-05 10:25:00", "Type": "SELL", "Amount": 400, "BrokerCode": "ABX"}
trade_summary
start
end
TotalBuy
TotalSell
2019-02-05T10:00:00Z
2019-02-05T10:15:00Z
800
0
2019-02-05T10:15:00Z
2019-02-05T10:30:00Z
600
400
2019-02-05T10:30:00Z
2019-02-05T10:45:00Z
900
0
2019-02-05T10:45:00Z
2019-02-05T11:00:00Z
0
500